

Тестирование веб-приложений

Основы тестирования: виды тестов, TDD и инструменты Python

Зачем нужно тестирование

Любое приложение должно быть надёжным, соответствовать требованиям и легко изменяться, когда требования меняются. Один разработчик может удержать это в голове, но когда над проектом работает команда, без тестов становится сложно быть уверенным, что новые изменения ничего не сломали.

Разные виды тестов решают разные задачи:

- приёмочные тесты** — подтверждают, что приложение работает так, как нужно заказчику;
- модульные тесты (unit-тесты)** — дают уверенность, что новые изменения в коде не ломают существующую логику;
- интеграционные тесты** — подтверждают, что приложение корректно взаимодействует с внешними системами: базами данных, API, сторонними сервисами.

Виды тестирования

Тесты бывают ручными (человек проверяет вручную) и автоматизированными (проверку выполняет код). Основные виды:

Вид тестирования	Что проверяет
Юнит-тестирование (модульное)	Отдельные компоненты приложения — функции и методы
Интеграционное	Как отдельные компоненты работают вместе (БД, сервер, клиент)
Функциональное	Соответствие приложения требованиям и документации
Тестирование производительности	Скорость и эффективность работы под нагрузкой
Тестирование безопасности	Защищённость от уязвимостей и атак
Тестирование на совместимость	Работа в разных браузерах, ОС (операционных системах) и устройствах
Регрессионное	Не сломали ли новые изменения уже существующий функционал
Сквозное (E2E, End-to-End)	Приложение целиком — все сценарии использования от начала до конца

Минимум для любого проекта

Юнит-тесты и функциональное тестирование — это база, без которой не стоит обходиться. Юнит-тесты гарантируют, что отдельные части кода работают правильно, а функциональное тестирование — что приложение в целом соответствует требованиям. Остальные виды важны, но нужны не всегда: зависит от конкретного проекта.

Интеграционное vs функциональное

Оба вида дают общую картину того, как приложение работает, но проверяют разное:

- Интеграционное** — работают ли компоненты *вместе* корректно (например, связка БД + сервер + клиент);
- Функциональное** — делает ли приложение то, что от него ожидают пользователи (вход в систему, добавление и редактирование данных и т. д.).

Главная цель любых тестов — уверенность: что приложение работает без ошибок, что проблемы находят и исправляют до релиза, и что итоговый продукт надёжный и качественный.

Разработка через тестирование (TDD)

TDD (Test-Driven Development) — подход, при котором тест на нужную функциональность пишут до самого кода. Это заставляет заранее чётко продумать, как код должен себя вести.

ШАГ 1

Red

Пишем тест на функциональность, которой ещё нет. Тест не проходит — кода-то ещё нет.

ШАГ 2

Green

Пишем минимум кода, чтобы тест прошёл. Код может быть неидеальным — главное, чтобы работал.

ШАГ 3

Refactor

Улучшаем структуру и качество кода, не меняя поведение. Все тесты должны остаться зелёными.

Плюсы

- Код получается более модульным и гибким
- Тесты работают как живая документация
- Ошибки находят на раннем этапе
- Изменения в коде безопаснее вносить

Минусы

- Есть кривая обучения — нужна практика
- Поначалу разработка кажется медленнее
- Тесты тоже приходится обновлять при изменении требований
- Риск переусердствовать с лишними тестами

Пакеты для тестирования в Python

Инструмент	Назначение
unittest	Встроенный в Python фреймворк для юнит-тестов
pytest	Более гибкий и мощный фреймворк, чем unittest
nose	Расширяет возможности unittest
coverage	Измеряет, какой процент кода покрыт тестами
Selenium	Автоматизация тестирования веб-интерфейса через браузер

На заметку

Идеального инструмента на все случаи не существует. Типичная связка: **unittest** или **pytest** — для юнит-тестов, **Selenium** — для автоматизации тестирования веб-интерфейса.

Фреймворк Unittest

Модульное тестирование

Модульное (юнит) тестирование — проверка отдельных модулей и компонентов кода на предмет ошибок. Такие тесты пишут на ранних этапах разработки, ещё до объединения кода с остальным приложением.

Что даёт модульное тестирование:

- уверенность, что отдельные компоненты кода работают правильно;
- меньше ошибок в готовом коде;
- более быструю разработку — изменения проверяются автоматически, без ручного тестирования всего приложения.

unittest — фреймворк для модульного тестирования, встроенный в ядро Python. Поддерживает все возможности языка и не требует установки сторонних пакетов.

Пример теста

```
import unittest

def multiply_numbers(x, y):
    return x * y

class TestMultiplication(unittest.TestCase):
    def test_multiply_two_positive_numbers(self):
        result = multiply_numbers(2, 3)
        self.assertEqual(result, 6)

    def test_multiply_positive_and_negative_numbers(self):
        result = multiply_numbers(2, -3)
        self.assertEqual(result, -6)

    def test_multiply_two_negative_numbers(self):
        result = multiply_numbers(-2, -3)
        self.assertEqual(result, 6)

if __name__ == '__main__':
    unittest.main()
```

Правила написания теста:

- класс должен наследоваться от `unittest.TestCase` ;
- имя класса должно начинаться с `Test` — иначе `unittest` его проигнорирует;
- каждый метод-тест должен начинаться с `test_` , чтобы фреймворк распознал его как тест;
- сравнение ожидаемого и фактического результата выполняется через методы вида `assertEqual` .

Fail vs Error

По итогам запуска все тесты делятся на 3 группы: пройден успешно, провален (fail) и вызвавший ошибку (error). Провал теста означает неправильное поведение приложения. Ошибка — это необработанное исключение (`Exception`) в коде, из-за которого приложение «падает».

Запуск тестов

Тесты можно запускать прямо из кода вызовом `unittest.main()` , но чаще их запускают из терминала — например, для интеграции с CI/CD (Continuous Integration / Continuous Deployment — непрерывная интеграция и доставка кода):

Команда	Что делает
<code>python -m unittest test_file.py</code>	Запускает все тесты в файле
<code>python -m unittest test_file.TestClass.test_method</code>	Запускает один конкретный тест
<code>python -m unittest discover test_directory</code>	Запускает все тесты в директории
<code>python -m unittest -v test_file.py</code>	Подробный (verbose) вывод — с именем каждого теста

Методы проверки (assert)

Класс `TestCase` предоставляет набор методов для проверки условий:

Метод	Проверяет
<code>assertEqual(a, b)</code>	а и b равны
<code>assertNotEqual(a, b)</code>	а и b не равны
<code>assertTrue(x)</code>	х истинно (не False, 0, "", None)
<code>assertFalse(x)</code>	х ложно
<code>assertIs(a, b)</code>	а и b — один и тот же объект
<code>assertIsNot(a, b)</code>	а и b — разные объекты
<code>assertIsNone(x)</code>	х равно None
<code>assertIsNotNone(x)</code>	х не равно None
<code>assertRaises(...)</code>	код внутри должен вызвать указанное исключение

Test fixture

Test fixture — подготовка окружения перед тестами и его очистка после них. Например, создание временной базы данных перед тестами или подготовка мок-объектов (заглушек — разберём дальше).

Для этого в unittest есть методы:

- `setUp` / `tearDown` — выполняются до и после **каждого** теста;
- `setUpClass` / `tearDownClass` — выполняются один раз до и после **всех** тестов класса (объявляются как `@classmethod`).

```
import unittest

class MyTest(unittest.TestCase):
    def setUp(self):
        self.my_list = [1, 2, 3]

    def tearDown(self):
        del self.my_list

    def test_list_length(self):
        self.assertEqual(len(self.my_list), 3)

    def test_list_contents(self):
        self.assertEqual(self.my_list, [1, 2, 3])

if __name__ == '__main__':
    unittest.main()
```

Порядок выполнения хорошо виден на примере с классовыми методами. Есть модуль с простыми вычислениями:

```
# ops.py
def add(a, b):
    return a + b

def sub(a, b):
    return a - b

def mul(a, b):
    return a * b

def div(a, b):
    return a / b
```

И набор тестов для него:

```
# test_ops.py
import unittest
from ops import add, sub, mul, div

class TestExamples(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        print('Перед всеми тестами')

    @classmethod
    def tearDownClass(cls):
        print('После всех тестов')

    def setUp(self):
        print('Перед тестом')

    def tearDown(self):
        print('После теста')

    def test_add(self):
        self.assertEqual(add(2, 3), 5)

    def test_sub(self):
        self.assertEqual(sub(2, 3), -1)

    def test_mul(self):
        self.assertEqual(mul(2, 3), 6)

    def test_div(self):
        self.assertAlmostEqual(div(2, 3), 0.66666666)
        with self.assertRaises(ZeroDivisionError):
            div(3, 0)

if __name__ == '__main__':
    unittest.main()
```

Порядок выполнения

setUpClass выполняется один раз перед всеми тестами класса, затем для каждого теста: setUp → сам тест → tearDown, и в самом конце один раз tearDownClass. Для сравнения дробных чисел используется assertAlmostEqual (сравнение с округлением), а assertRaises как контекстный менеджер (with) проверяет, что код внутри блока вызывает нужное исключение.

Тестирование асинхронного кода

Для тестирования асинхронного (async) кода в unittest есть класс `IsolatedAsyncioTestCase` — тестовые методы в нём тоже пишутся как `async` и вызываются через `await`.

```
import asyncio
import unittest

async def async_add(a, b):
    await asyncio.sleep(1)
    return a + b

class TestAsyncMethod(unittest.IsolatedAsyncioTestCase):
    async def test_add(self):
        result = await async_add(2, 3)
        self.assertEqual(result, 5)

if __name__ == '__main__':
    unittest.main()
```

Mock-объекты и заглушки

Mock-объект — объект-имитатор. Он изолирует тестируемый код от внешних зависимостей, заменяя их на полностью контролируемую имитацию.

Mock-объекты нужны, когда:

- код обращается к внешним ресурсам — базам данных, веб-сервисам, файловой системе;
- код зависит от компонентов, которые ещё не реализованы;
- нужно воспроизвести ситуацию, которую сложно создать в реальности, — например, конкретное исключение.

```
from unittest.mock import MagicMock
from my_module import ProductionClass

thing = ProductionClass()
thing.method = MagicMock(return_value=3)
thing.method(1, 2, 3, key='value')

thing.method.assert_called_with(1, 2, 3, key='value')
```

Метод `MagicMock(return_value=3)` задаёт, что должен вернуть mock при вызове. А `assert_called_with(...)` проверяет, что mock был вызван именно с такими аргументами.

Чтобы временно подменить метод или класс «на горячую», используется `patch` — как менеджер контекста или как декоратор:

```
with patch.object(ProductionClass, 'method', return_value=3) as mock_method:
    thing = ProductionClass()
    thing.method(1, 2, 3)
    mock_method.assert_called_once_with(1, 2, 3)
```

Так же можно подменить сразу несколько классов в модуле:

```

from unittest.mock import patch

@patch('module.ClassName2')
@patch('module.ClassName1')
def test_class(MockClass1, MockClass2):
    module.ClassName1()
    module.ClassName2()
    assert MockClass1.called
    assert MockClass2.called

```

Декораторы `@patch` подменяют классы в модуле на mock-объекты и передают их в тест как параметры функции. Атрибут `called` показывает, был ли mock вызван.

side_effect задаёт более сложное поведение mock-объекта, чем просто фиксированный `return_value` :

- функция — будет вызываться при каждом обращении к mock-объекту;
- класс исключения — mock-объект будет выбрасывать это исключение при каждом вызове;
- список (или другой итерируемый объект) — при каждом вызове будет возвращаться следующее значение из него.

```

def raise_error(*args, **kwargs):
    raise ValueError("Test error")

mock_obj = Mock(side_effect=raise_error)

```

```

with patch("builtins.open", side_effect=FileNotFoundError):
    # код теста
    ...

```

```

mock_obj = Mock(side_effect=[1, 2, 3])
print(mock_obj()) # 1
print(mock_obj()) # 2
print(mock_obj()) # 3

```

Приоритет

Если заданы одновременно `side_effect` и `return_value`, приоритет у `side_effect`. Значение `return_value` будет использовано, только когда `side_effect` исчерпан или равен `None`.

Практический пример: поиск рецепта в файле

Есть CSV-файл (Comma-Separated Values — значения, разделённые запятыми) с рецептами. Каждая строка: идентификатор рецепта, затем через запятую название, затем через запятую ингредиенты:

```

60b90c1c13067a15887e1ae1,Herbed Baked Salmon,4 lemons,1 large red onion,2 tablespoons chopped fresh basil
60b90c3b13067a15887e1ae4,Watermelon Cucumber Salad,1 large seedless watermelon,12 leaves fresh mint,1 cup crumbled fe

```

Нужна функция `get_recipe(path, search_id)`, которая через контекстный менеджер `with` открывает файл, ищет строку с нужным id и возвращает словарь вида `{"id": ..., "name": ..., "ingredients": [...]}`. Если рецепта нет — вернуть `None`.

```
def get_recipe(path, search_id):
    result = None
    with open(path, "r") as f:
        for line in f:
            (id, name, *recipes) = line.strip().split(",")
            if id == search_id:
                result = {"id": id, "name": name, "ingredients": recipes}
    return result
```

Тестирование с mock_open

Чтобы не создавать реальный файл на диске, unittest предлагает `mock_open` — он имитирует поведение открытого файла и «скармливает» тесту заранее подготовленную строку вместо настоящего чтения с диска.

```
import unittest
from unittest.mock import patch, mock_open

class TestGetRecipe(unittest.TestCase):
    mock_data = (
        "60b90c1c13067a15887e1ae1,Herbed Baked Salmon,4 lemons,1 large red onion,2 tablespoons chopped fresh basil\n"
        "60b90c2413067a15887e1ae2,Lemon Pancakes,2 tablespoons baking powder,1 cup vanilla-flavored almond milk,1 lem"
        "60b90c2e13067a15887e1ae3,Chicken and Cold Noodles,6 ounces dry Chinese noodles,1 tablespoon sesame oil,3 tab"
    )

    @classmethod
    def setUpClass(cls):
        cls.mock_open_file = mock_open(read_data=cls.mock_data)

    @classmethod
    def tearDownClass(cls):
        cls.mock_open_file = None

    def test_get_first_recipe(self):
        uuid = "60b90c1c13067a15887e1ae1"
        with patch("builtins.open", self.mock_open_file):
            result = get_recipe("fake.csv", uuid)
            self.assertEqual("Herbed Baked Salmon", result.get("name"))

    def test_get_nonexistent_recipe(self):
        with patch("builtins.open", self.mock_open_file):
            result = get_recipe("fake.csv", "nonexistent_id")
            self.assertIsNone(result)

    def test_empty_file(self):
        empty_mock = mock_open(read_data="")
        with patch("builtins.open", empty_mock):
            result = get_recipe("empty.csv", "60b90c1c13067a15887e1ae1")
            self.assertIsNone(result)

    def test_file_not_found(self):
        with patch("builtins.open", side_effect=FileNotFoundError):
            with self.assertRaises(FileNotFoundError):
                get_recipe("nonexistent_file.csv", "60b90c1c13067a15887e1ae1")
```

- `setUpClass` / `tearDownClass` — создают и очищают общий `mock_open_file` один раз для всего класса, чтобы не готовить тестовые данные заново в каждом тесте;
- `test_get_first_recipe` (и аналогичный тест для второго рецепта) — подменяет `open` на мок и проверяет, что функция верно достаёт id, название и ингредиенты из нужной строки файла;
- `test_get_nonexistent_recipe` — ищет несуществующий id и проверяет, что функция возвращает `None` ;

- `test_empty_file` — отдельный мок с пустыми данными (`read_data=""`) — тоже должен дать `None` ;
- `test_file_not_found` — `patch(..., side_effect=FileNotFoundError)` имитирует отсутствие файла на диске. Тест проверяет, что функция **не глотает** исключение, а даёт ему «всплыть» наружу — это и есть ожидаемое поведение.

Фреймворк PyTest

PyTest — один из самых популярных фреймворков для тестирования в Python. В отличие от `unittest`, он делает упор на простоту: тесты пишут как обычные функции, без обязательных классов. PyTest умеет запускать и тесты, написанные на `unittest` и `nose`, а также даёт подробный, понятный вывод при падении теста.

```
poetry add pytest
```

Создание тестов

Файлы с тестами называют с префиксом `test_` или суффиксом `_test` . Тестовые функции тоже должны начинаться с `test_` — иначе `pytest` их не найдёт.

```
def inc(x):  
    return x + 1  
  
def test_answer():  
    assert inc(3) == 5
```

Запуск: `pytest main.py` . Если тест не проходит, PyTest подробно покажет, какое значение получилось и какое ожидалось:

```
main.py::test_answer FAILED  
  
    def test_answer():  
>         assert inc(3) == 5  
E         assert 4 == 5  
  
1 failed in 0.10s
```

Отличие от unittest

PyTest считает тест провальным при любом `AssertionError` — в отличие от `unittest`, здесь нет отдельного механизма, который бы отличал «неверное поведение» приложения от «ошибки» в самом коде теста.

Запуск тестов

Тесты запускают командой `pytest` из терминала, указывая файл или директорию: `pytest test_example.py` или `pytest tests/` . Полезные флаги:

Флаг	Что делает
<code>-v</code> / <code>--verbose</code>	Подробный вывод — имена всех тестов
<code>-s</code>	Не перехватывать вывод — <code>print</code> попадёт в консоль
<code>-x</code>	Остановиться на первой же ошибке или провале
<code>-q</code> / <code>--quiet</code>	Меньше сообщений о прогрессе
<code>-h</code> / <code>--help</code>	Справка по командам

Чтобы `pytest` мог правильно импортировать модули проекта, в `pyproject.toml` добавляют:

```
[tool.pytest.ini_options]
pythonpath = ["."]
```

Assert в PyTest

PyTest использует обычный оператор `assert` вместо методов вроде `assertEqual` — тесты получаются короче и читаются как обычный код:

```
import pytest

def test_basic_assertions():
    assert 1 == 1           # равенство
    assert 3 > 2           # истинность
    assert "h" in "hello"  # вхождение в коллекцию
    assert isinstance(1, int) # проверка типа

    with pytest.raises(ZeroDivisionError):
        1 / 0               # проверка исключения

    assert pytest.approx(0.1 + 0.2) == 0.3 # приблизительное равенство float
    assert len([1, 2, 3]) == 3             # длина коллекции

def test_exception_message():
    with pytest.raises(ValueError) as exc_info:
        raise ValueError("Неверное значение")
    assert str(exc_info.value) == "Неверное значение"
```

При падении `assert` PyTest сам покажет, какое значение получилось и какое ожидалось — не нужно, как в unittest, подбирать отдельный метод под конкретную проверку.

Fixture — начальные условия тестов

Fixture (англ. «крепление») — код, который готовит окружение перед тестом и, если нужно, убирает за собой после. Чтобы не повторять подготовку данных в каждом тесте, фикстуру объявляют один раз через декоратор `@pytest.fixture`, а затем просто указывают её имя как параметр нужного теста.

```
import pytest

@pytest.fixture
def some_data():
    return {"a": 1, "b": 2, "c": 3}

def test_addition(some_data):
    assert some_data["a"] + some_data["b"] == 3

def test_subtraction(some_data):
    assert some_data["c"] - some_data["b"] == 1
```

Важно

Фикстура передаётся в тест как аргумент с тем же именем, что и сама фикстура. Ошибётесь в имени — pytest выбросит исключение.

Чаще всего фикстуры используют для подготовки внешних ресурсов — например, соединения с базой данных (БД):

```
# db.py
import sqlite3
from contextlib import contextmanager

@contextmanager
def create_connection():
    conn = sqlite3.connect(':memory:')
    yield conn
    conn.rollback()
    conn.close()
```

```
import pytest
import db

@pytest.fixture
def db_connection():
    with db.create_connection() as connection:
        yield connection

def test_db_operation(db_connection):
    cursor = db_connection.cursor()
    cursor.execute("CREATE TABLE users (id INTEGER PRIMARY KEY, fullname TEXT, email TEXT)")
    cursor.execute("INSERT INTO users (fullname, email) VALUES (?, ?)", ("John", "john@example.com"))
    db_connection.commit()
    r = cursor.execute("SELECT fullname, email FROM users")
    assert r.fetchone() == ("John", "john@example.com")
```

Области действия (scope) фикстур

scope	Когда пересоздаётся
<code>function</code> (по умолчанию)	Заново для каждого теста
<code>class</code>	Один раз на класс тестов
<code>module</code>	Один раз на файл с тестами
<code>session</code>	Один раз на весь запуск тестов

yield вместо return

Если в фикстуре использовать `yield` вместо `return`, всё, что написано после `yield`, выполнится уже **после** теста — удобно для очистки ресурсов: закрыть соединение, удалить временные файлы и т. д. Например, при `scope="module"` фикстура создаётся один раз перед первым тестом файла и уничтожается после последнего, а фикстура с `scope="function"` пересоздаётся перед каждым тестом. Чтобы увидеть вывод `print` из фикстур, тесты нужно запускать с флагом `-s`.

Параметризованные тесты

Декоратор `@pytest.mark.parametrize` запускает один и тот же тест с разными наборами входных данных — не нужно копировать тест под каждый случай.

```
import pytest

@pytest.mark.parametrize("a, b, result", [
    (1, 2, 3),
    (2, 3, 5),
    (3, 4, 7),
])
def test_addition(a, b, result):
    assert a + b == result
```

PyTest запустит тест отдельно для каждой строки списка — в примере получится 3 теста. Параметризацию можно сочетать с фикстурами:

```
@pytest.fixture
def add_fixture():
    return lambda x, y: x + y

@pytest.mark.parametrize('a, b, expected', [
    (1, 2, 3),
    (-2, 5, 3),
])
def test_addition(add_fixture, a, b, expected):
    assert add_fixture(a, b) == expected
```

А можно параметризовать саму фикстуру — тогда все тесты, которые её используют, тоже запустятся несколько раз:

```
@pytest.fixture(params=[1, 2, 3])
def numbers(request):
    return request.param

def test_multiplication(numbers):
    assert numbers * 2 < 7
```

Mock-объекты

Как и unittest, PyTest можно использовать вместе с `unittest.mock`:

```
import requests
from unittest.mock import Mock, patch

def send_request(url):
    response = requests.get(url)
    return response.status_code

def test_send_request():
    mock_get = Mock(return_value=Mock(status_code=200))
    with patch("requests.get", mock_get):
        status_code = send_request("http://example.com")
        assert status_code == 200
```

Более удобный вариант — плагин `pytest-mock` (`poetry add pytest-mock`). Он даёт фикстуру `mockers`, которая сама следит, чтобы после теста всё вернулось на место:

```
def test_send_request(mockers):
    mock_get = mockers.patch('requests.get')
    mock_get.return_value.status_code = 200
    status_code = send_request('http://example.com')
    assert status_code == 200
    mock_get.assert_called_once_with('http://example.com')
```

Так же можно подготовить и более сложный ответ — например, JSON (JavaScript Object Notation — текстовый формат обмена данными):

```
def test_get_user_data(mock):
    mock_get = mock.patch('requests.get')
    mock_response = mock.Mock()
    mock_response.json.return_value = {"id": 1, "name": "John Doe"}
    mock_get.return_value = mock_response

    result = get_user_data(1)

    mock_get.assert_called_once_with("https://api.example.com/users/1")
    assert result == {"id": 1, "name": "John Doe"}
```

`mock.spy` позволяет «шпионить» за настоящим методом — он продолжает работать как обычно, но фиксирует, вызывали ли его, сколько раз и с какими аргументами:

```
def test_data_processor(mock):
    processor = DataProcessor()
    spy = mock.spy(processor, "process_data")

    result = processor.process_data("hello")

    assert spy.called
    assert spy.call_count == 1
    assert result == "HELLO"
```

А через `mock.patch.object(..., new_callable=mock.PropertyMock)` можно подменить даже `@property` класса.

Тестирование асинхронного кода

Для асинхронного (async) кода нужен плагин `pytest-asyncio` (`poetry add pytest-asyncio`) и декоратор `@pytest.mark.asyncio` :

```
import asyncio
import pytest

async def fetch_data(url):
    await asyncio.sleep(1)
    return f"Данные из {url}"

@pytest.mark.asyncio
async def test_fetch_data():
    result = await fetch_data("https://example.com")
    assert result == "Данные из https://example.com"
```

Фикстуры и параметризация точно так же работают с асинхронными тестами:

```

@pytest.fixture
async def async_data():
    await asyncio.sleep(0.5)
    return "Асинхронные данные"

@pytest.mark.asyncio
async def test_with_async_fixture(async_data):
    assert await async_data == "Асинхронные данные"

@pytest.mark.asyncio
@pytest.mark.parametrize("url", ["https://example.com", "https://test.com"])
async def test_fetch_multiple(url):
    result = await fetch_data(url)
    assert result == f"Данные из {url}"

```

Чтобы проверить, что асинхронная функция укладывается в отведённое время, используют `asyncio.wait_for` с таймаутом. Если функция не успела — вылетит `asyncio.TimeoutError`, что тоже можно протестировать через `pytest.raises`:

```

@pytest.mark.asyncio
async def test_timeout():
    with pytest.raises(asyncio.TimeoutError):
        await asyncio.wait_for(asyncio.sleep(1), timeout=0.5)

```

Практический пример: тесты `get_recipe` на PyTest

Перепишем тесты для `get_recipe` из предыдущего раздела с `unittest` на `pytest`. Основные изменения:

- вместо `setUpClass` / `tearDownClass` — фикстуры;
- вместо двух почти одинаковых тестов (`test_get_first_recipe` , `test_get_second_recipe`) — один параметризованный;
- вместо `self.assertEqual` / `self.assertIsNone` — обычный `assert` ;
- вместо `self.assertRaises` — `with pytest.raises(...)` .

```

import pytest
from unittest.mock import patch, mock_open

def get_recipe(path, search_id):
    result = None
    with open(path, "r") as f:
        for line in f:
            (id, name, *recipes) = line.strip().split(",")
            if id == search_id:
                result = {"id": id, "name": name, "ingredients": recipes}
    return result

@pytest.fixture
def mock_recipes_data():
    return (
        "60b90c1c13067a15887e1ae1,Herbed Baked Salmon,4 lemons,1 large red onion,2 tablespoons chopped fresh basil\n",
        "60b90c2413067a15887e1ae2,Lemon Pancakes,2 tablespoons baking powder,1 cup vanilla-flavored almond milk,1 lemon\n"
    )

@pytest.fixture
def mock_file(mock_recipes_data):
    return mock_open(read_data=mock_recipes_data)

@pytest.mark.parametrize(
    "uuid,expected_name,expected_ingredients",
    [
        (
            "60b90c1c13067a15887e1ae1",
            "Herbed Baked Salmon",
            ["4 lemons", "1 large red onion", "2 tablespoons chopped fresh basil"],
        ),
        (
            "60b90c2413067a15887e1ae2",
            "Lemon Pancakes",
            ["2 tablespoons baking powder", "1 cup vanilla-flavored almond milk", "1 lemon"],
        ),
    ],
)

def test_get_existing_recipe(uuid, expected_name, expected_ingredients, mock_file):
    with patch("builtins.open", mock_file):
        result = get_recipe("fake.csv", uuid)
        assert result["id"] == uuid
        assert result["name"] == expected_name
        assert result["ingredients"] == expected_ingredients

def test_get_nonexistent_recipe(mock_file):
    with patch("builtins.open", mock_file):
        result = get_recipe("fake.csv", "nonexistent_id")
        assert result is None

def test_empty_file():
    empty_mock = mock_open(read_data="")
    with patch("builtins.open", empty_mock):
        result = get_recipe("empty.csv", "60b90c1c13067a15887e1ae1")
        assert result is None

def test_file_not_found():
    with patch("builtins.open", side_effect=FileNotFoundError):
        with pytest.raises(FileNotFoundError):
            get_recipe("nonexistent_file.csv", "60b90c1c13067a15887e1ae1")

```

Фикстура `mock_recipes_data` готовит тестовые данные, а `mock_file` принимает её как параметр и на её основе создаёт объект `mock_open` — фикстуры в `pytest` могут опираться друг на друга. Параметризованный `test_get_existing_recipe` заменяет собой сразу два теста из `unittest`-версии.

Документирование кода проекта

Хорошая документация не менее важна, чем сам API (Application Programming Interface — программный интерфейс приложения): его удобство лишь наполовину зависит от набора функций и классов, вторая половина — насколько легко в них разобраться *без чтения исходного кода*.

Способы документирования в Python:

- **комментарии** — текст вне кода, который поясняет, что он делает; интерпретатор их игнорирует;
- **docstrings** — специальный комментарий в тройных кавычках для модулей, классов, функций и методов; доступен через атрибут `__doc__`;
- **Sphinx** — инструмент, который генерирует документацию «профессионального» вида на основе кода и docstrings.

Стили docstring

Есть два распространённых стиля. **reStructuredText** — каждый параметр и возвращаемое значение описываются отдельной строкой с ключевым словом (`:param` , `:type` , `:return` , `:rtype`):

```
def add(a: int, b: int) -> int:
    """
    Возвращает сумму двух чисел.

    :param a: Первое число.
    :type a: int
    :param b: Второе число.
    :type b: int
    :return: Сумма a и b.
    :rtype: int
    """
    return a + b
```

Google-стиль — читается более естественно, часто выбирают именно его ради лаконичности:

```
def add(a: int, b: int) -> int:
    """
    Возвращает сумму двух чисел.

    Args:
        a (int): Первое число.
        b (int): Второе число.

    Returns:
        int: Сумма a и b.
    """
    return a + b
```

Какой стиль выбрать

reStructuredText удобнее для крупных проектов со строгой типизацией и автогенерацией документации через Sphinx. Google-стиль — более естественный и читабельный для людей. Если важнее лаконичность и удобство чтения, чаще выбирают именно Google-стиль.

PyDoc

PyDoc — встроенный модуль для чтения документации модулей и пакетов. Он берёт текст из атрибута `__doc__`, из docstring сразу после объявления класса/функции и из строки-комментария в самом начале модуля.

```
python -m pydoc [имя_модуля]
```

Команда `pydoc -b`, запущенная в проекте, поднимает локальный сервер и открывает документацию в браузере.

Sphinx

Sphinx — более мощный инструмент для генерации документации. Использует разметку reStructuredText и умеет автоматически собирать документацию из docstrings в форматах HTML, PDF, EPUB и LaTeX.

Установка как dev-зависимости (нужна только при разработке, не в проде):

```
poetry add sphinx -G dev
```

Команда	Что делает
<code>sphinx-quickstart</code>	Создаёт новый проект документации: задаёт вопросы (название, автор, язык) и генерирует структуру файлов
<code>sphinx-build</code>	Собирает документацию из исходников в HTML/PDF/LaTeX и другие форматы
<code>sphinx-apidoc</code>	Автоматически генерирует документацию по модулям Python
<code>sphinx-autogen</code>	Генерирует документацию на основе шаблонов

Инициализация в корне проекта создаёт директорию `docs`:

```
sphinx-quickstart docs
```

В файле `docs/conf.py` задаются настройки сборки:

```
import sys
import os

sys.path.append(os.path.abspath(".")) # чтобы Sphinx видел модули проекта

project = "Rest API"
copyright = "2024, krabat"
author = "krabat"

extensions = ["sphinx.ext.autodoc"] # собирает документацию из docstrings

templates_path = ["_templates"]
exclude_patterns = ["_build"]

html_theme = "nature"
html_static_path = ["_static"]
```

В файле `docs/index.rst` перечисляют, для каких модулей проекта нужно собрать документацию:

```

Rest API documentation
=====

.. toctree::
    :maxdepth: 2

REST API repository Notes
=====

.. automodule:: src.repository.notes
    :members:
    :undoc-members:
    :show-inheritance:

```

Директива `automodule` указывает путь к модулю (как в обычном `import`). Рядом с ней:

- `:members:` — включить в документацию все функции и классы модуля;
- `:undoc-members:` — включить и те, у которых нет docstring;
- `:show-inheritance:` — показать наследование классов.

Перед сборкой у нужных функций и методов должны быть docstrings. Пример для пары методов репозитория заметок:

```

class NoteRepository:
    def __init__(self, session: AsyncSession):
        """
        Args:
            session: Подключённая сессия к базе данных.
        """
        self.db = session

    async def get_notes(self, skip: int, limit: int, user: User) -> List[Note]:
        """
        Возвращает список заметок пользователя с пагинацией.

        Args:
            skip: Сколько заметок пропустить.
            limit: Максимум заметок в ответе.
            user: Владелец заметок.

        Returns:
            Список заметок.
        """
        ...

```

Остальные методы документируют аналогично. Дальше — сборка (Windows: `.\make.bat html`, Linux/macOS: `make html`). Результат появится в `docs/_build/html/index.html`.

Частые ошибки

Нарушение синтаксиса reStructuredText в `index.rst` (например, линия из `=` под заголовком короче самого заголовка) и параметры конфигурации без значения по умолчанию — оба ломают сборку.

Зачем вообще нужна документация:

- упрощает использование ПО (программного обеспечения) — не нужно читать исходный код, чтобы понять, как им пользоваться;
- ускоряет поддержку проекта — быстрее находить ответы на вопросы;
- улучшает качество кода — чтобы описать код, разработчик сам лучше в нём разбирается;
- ускоряет разработку — код, документированный по ходу дела, проще организовать и понять позже.

Модульные тесты для репозитория

Модульные тесты можно писать для любого слоя приложения — репозиториев, сервисов, маршрутов (роутов). Для репозиториев (слой, который отвечает за обращение к базе данных) главная цель — убедиться, что все запросы к БД (базе данных) работают правильно.

Тестовые файлы принято называть с суффиксом `unit`, чтобы сразу было понятно, что это именно модульные тесты, например: `tests/test_tag_repository_unit.py`, `tests/test_note_repository_unit.py`. Для асинхронного кода `pytest` удобнее `unittest`, поэтому дальше используем именно его.

Фикстуры и мок сессии

Чтобы не обращаться к настоящей базе, сессию `SQLAlchemy` подменяют мок-объектом `AsyncMock`:

```
import pytest
from unittest.mock import AsyncMock, MagicMock
from sqlalchemy.ext.asyncio import AsyncSession

from src.repository.tags import TagRepository

@pytest.fixture
def mock_session():
    return AsyncMock(spec=AsyncSession)

@pytest.fixture
def tag_repository(mock_session):
    return TagRepository(mock_session)

@pytest.fixture
def user():
    return User(id=1, username="testuser")
```

Зачем нужен `spec=AsyncSession`

Параметр `spec` задаёт мок-объекту «эталон»: `AsyncMock` будет имитировать только те методы и атрибуты, которые реально есть у `AsyncSession` (`execute`, `commit`, `refresh` и т. д.). Это защищает от опечаток — обращение к несуществующему методу мока сразу даст ошибку, как и с настоящим объектом.

Примеры тестов

Тест принято делить на три блока: `# Setup` — подготовка мок-результата, `# Call method` — вызов тестируемого метода, `# Assertions` — проверки.

```
@pytest.mark.asyncio
async def test_get_tags(tag_repository, mock_session, user):
    # Setup
    mock_result = MagicMock()
    mock_result.scalars.return_value.all.return_value = [Tag(id=1, name="test tag", user=user)]
    mock_session.execute = AsyncMock(return_value=mock_result)

    # Call method
    tags = await tag_repository.get_tags(skip=0, limit=10, user=user)

    # Assertions
    assert len(tags) == 1
    assert tags[0].name == "test tag"
```

```

@pytest.mark.asyncio
async def test_create_tag(tag_repository, mock_session, user):
    # Setup
    tag_data = TagModel(name="new tag")

    # Call method
    result = await tag_repository.create_tag(body=tag_data, user=user)

    # Assertions
    assert isinstance(result, Tag)
    assert result.name == "new tag"
    mock_session.add.assert_called_once()
    mock_session.commit.assert_awaited_once()
    mock_session.refresh.assert_awaited_once_with(result)

```

В `test_get_tags` мок настроен так, чтобы `scalars().all()` вернул список из одного тега — так имитируется результат запроса к базе. В `test_create_tag` проверяется не только результат, но и то, что были вызваны нужные методы сессии: `add` (синхронный) и `commit` / `refresh` (асинхронные — отсюда разница между `assert_called_once()` и `assert_awaited_once()`).

Метод	Проверяет
<code>assert_called_once()</code>	Метод вызван ровно один раз
<code>assert_called_once_with(...)</code>	Вызван один раз и с такими аргументами
<code>assert_awaited_once()</code>	То же самое, но для асинхронного метода (только у <code>AsyncMock</code>)
<code>assert_awaited_once_with(...)</code>	Асинхронный метод вызван один раз с такими аргументами
<code>assert_not_called()</code>	Метод вообще не вызывался
<code>assert_called()</code>	Метод вызывался хотя бы раз
<code>assert_called_with(...)</code>	Хотя бы раз вызван с такими аргументами
<code>assert_any_call(...)</code>	Среди всех вызовов был хотя бы один с такими аргументами
<code>call_count</code>	Сколько раз метод был вызван (число, не метод проверки)

По этой же схеме пишутся тесты `test_get_tag_by_id` (мок настраивает `scalar_one_or_none()`, чтобы вернуть один тег — проверяем, что найден нужный `id` и `name`), `test_update_tag` (готовим существующий тег, вызываем обновление, проверяем новое имя и что `commit` / `refresh` вызваны) и `test_remove_tag` (проверяем, что вызваны `delete` и `commit` для найденного тега).

Тесты для NoteRepository

Для заметок (`NoteRepository`) фикстуры и подход к мокам полностью аналогичны — те же `mock_session`, `note_repository`, `user` и структура Setup / Call method / Assertions:

- `test_get_notes` — список заметок пользователя, проверяет `title` и `description`;
- `test_get_note_by_id` — поиск заметки по `id`;
- `test_create_note` — создание заметки, проверяет тип результата и его поля;
- `test_remove_note` — удаление, проверяет вызовы `delete` и `commit`;
- `test_update_note` — обновление полей заметки, проверяет `commit` и `refresh`;
- `test_update_status_note` — обновление статуса `done`.

Интеграционные тесты маршрутов

Если модульные тесты проверяют функции и классы изолированно, то интеграционные — как компоненты работают *вместе*: как маршруты (роуты) API взаимодействуют с базой данных и бизнес-логикой. Далее покроем интеграционными тестами аутентификацию (`src/api/auth.py`), теги (`src/api/tags.py`) и пользователей (`src/api/users.py`).

`conftest.py` — общие фикстуры

Файл `tests/conftest.py` — специальное имя, которое pytest находит автоматически. Фикстуры, объявленные в нём, доступны сразу во всех тестовых файлах проекта — не нужно определять их заново в каждом модуле.

```

import asyncio
import pytest
import pytest_asyncio
from fastapi.testclient import TestClient
from sqlalchemy.pool import StaticPool
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker

from main import app
from src.database.models import Base, User
from src.database.db import get_db
from src.services.auth import create_access_token, Hash

SQLALCHEMY_DATABASE_URL = "sqlite+aiosqlite:///./test.db"

engine = create_async_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False},
    poolclass=StaticPool,
)

TestingSessionLocal = async_sessionmaker(autoflush=False, expire_on_commit=False, bind=engine)

test_user = {"username": "deadpool", "email": "deadpool@example.com", "password": "12345678"}

@pytest.fixture(scope="module", autouse=True)
def init_models_wrap():
    async def init_models():
        async with engine.begin() as conn:
            await conn.run_sync(Base.metadata.drop_all)
            await conn.run_sync(Base.metadata.create_all)
        async with TestingSessionLocal() as session:
            hash_password = Hash().get_password_hash(test_user["password"])
            current_user = User(username=test_user["username"], email=test_user["email"],
                                hashed_password=hash_password, confirmed=True)
            session.add(current_user)
            await session.commit()

    asyncio.run(init_models())

@pytest.fixture(scope="module")
def client():
    async def override_get_db():
        async with TestingSessionLocal() as session:
            try:
                yield session
            except Exception:
                await session.rollback()
                raise

    app.dependency_overrides[get_db] = override_get_db
    yield TestClient(app)

@pytest_asyncio.fixture()
async def get_token():
    return await create_access_token(data={"sub": test_user["username"]})

```

- `init_models_wrap` — пересоздаёт таблицы в тестовой SQLite-базе (`drop_all` + `create_all`) и добавляет тестового пользователя. `scope="module", autouse=True` означает: выполнить один раз для всего модуля, автоматически, без явного упоминания в тестах;
- `client` — подменяет зависимость `get_db` на тестовую базу (`app.dependency_overrides[get_db] = override_get_db`) и возвращает `TestClient` — объект FastAPI (построен поверх `httpx`), который умеет отправлять HTTP-запросы (GET, POST, PUT, DELETE) приложению и проверять ответ, как настоящий клиент;

- `get_token` — создаёт JWT-токен (JSON Web Token — формат токена авторизации) для тестового пользователя, чтобы в тестах, где нужна авторизация, не логиниться заново каждый раз.

Изоляция от реальной БД

Тестовая база нужна, чтобы тесты не портили реальные данные и выполнялись в предсказуемом, «чистом» состоянии — она создаётся заново перед тестами модуля. Так как у фикстуры `client` тоже `scope="module"`, данные внутри одного тестового модуля сохраняются между тестами — например, пользователь, созданный в `test_signup`, будет доступен в следующих тестах того же файла.

Тесты аутентификации

В `test_integration_auth.py` проверяют: регистрацию (`test_signup`), повторную регистрацию с тем же email → код 409 (`test_repeat_signup`), вход без подтверждённого email (`test_not_confirmed_login`), успешный вход (`test_login`), вход с неверным паролем и неверным email (`test_wrong_password_login`, `test_wrong_username_login`).

```
from unittest.mock import Mock

user_data = {"username": "agent007", "email": "agent007@gmail.com", "password": "12345678"}

def test_signup(client, monkeypatch):
    mock_send_email = Mock()
    monkeypatch.setattr("src.api.auth.send_email", mock_send_email)
    response = client.post("api/auth/register", json=user_data)
    assert response.status_code == 201, response.text
    data = response.json()
    assert data["username"] == user_data["username"]
    assert "hashed_password" not in data
```

`monkeypatch` — встроенный механизм `pytest` для подмены объекта на время теста (переменной окружения, функции, класса). Здесь он подменяет реальную `send_email` на мок — чтобы тест не отправлял настоящее письмо.

```
@pytest.mark.asyncio
async def test_login(client):
    async with TestingSessionLocal() as session:
        current_user = await session.execute(select(User).where(User.email == user_data["email"]))
        current_user = current_user.scalar_one_or_none()
        if current_user:
            current_user.confirmed = True
            await session.commit()

    response = client.post("api/auth/login",
                          data={"username": user_data["username"], "password": user_data["password"]})
    assert response.status_code == 200, response.text
    assert "access_token" in response.json()
```

Тест вручную подтверждает email пользователя прямо в базе (`confirmed = True`), а затем логинится — обратите внимание, что для входа передаётся `data`, а не `json`: сервер ждёт тело в формате формы (`application/x-www-form-urlencoded`).

Настройка pyproject.toml

Если pytest выводит предупреждение про `asyncio_default_fixture_loop_scope`, добавьте в `pyproject.toml`:

```
[tool.pytest.ini_options]
pythonpath = "."
filterwarnings = "ignore::DeprecationWarning"
asyncio_default_fixture_loop_scope = "function"
```

Значение `"function"` означает: новый event loop (цикл событий асинхронности) для каждой тестовой функции — это изолирует тесты друг от друга. Глобально скрывать `DeprecationWarning` стоит с осторожностью — так легко пропустить важные предупреждения об изменениях в зависимостях; лучше указывать конкретный пакет, например `filterwarnings = "ignore::DeprecationWarning:jose.*:"`.

Тесты маршрутов тегов

В `test_integration_tags.py` проверяют полный CRUD (Create, Read, Update, Delete — базовые операции создания, чтения, обновления и удаления): создание тега, получение по id (включая «не найден» → `404`), получение списка, обновление (включая «не найден»), удаление и повторное удаление уже удалённого тега (тоже `404`). Каждый запрос идёт с токеном из фикстуры `get_token`:

```
def test_create_tag(client, get_token):
    response = client.post(
        "/api/tags",
        json={"name": "test_tag"},
        headers={"Authorization": f"Bearer {get_token}"},
    )
    assert response.status_code == 201, response.text
    data = response.json()
    assert data["name"] == "test_tag"
    assert "id" in data
```

Остальные тесты этого файла устроены так же — запрос к нужному маршруту с токеном, проверка кода ответа и содержимого JSON.

Тесты маршрутов пользователей

```
def test_get_me(client, get_token):
    headers = {"Authorization": f"Bearer {get_token}"}
    response = client.get("api/users/me", headers=headers)
    assert response.status_code == 200, response.text
    assert "avatar" in response.json()

@patch("src.services.upload_file.UploadFileService.upload_file")
def test_update_avatar_user(mock_upload_file, client, get_token):
    fake_url = "http://example.com/avatar.jpg"
    mock_upload_file.return_value = fake_url
    headers = {"Authorization": f"Bearer {get_token}"}
    file_data = {"file": ("avatar.jpg", b"fake image content", "image/jpeg")}

    response = client.patch("/api/users/avatar", headers=headers, files=file_data)

    assert response.status_code == 200, response.text
    assert response.json()["avatar"] == fake_url
    mock_upload_file.assert_called_once()
```

`test_get_me` проверяет, что маршрут возвращает данные авторизованного пользователя. В `test_update_avatar_user` декоратор `@patch` подменяет реальную загрузку файла в облако на мок с фиктивным URL — тест проверяет обновление

аватара, не обращаясь к настоящему сервису хранения файлов. MIME-тип `image/jpeg` (MIME — Multipurpose Internet Mail Extensions, стандарт указания типа содержимого файла) в `files` сообщает серверу, что за файл отправлен.

pytest-cov: покрытие кода тестами

pytest-cov — плагин, который показывает, какая часть кода реально выполняется при прогоне тестов (покрытие тестами), помогая найти протестированные участки.

```
poetry add pytest-cov
```

```
pytest --cov=src tests/
```

Команда	Формат отчёта
<code>pytest --cov=src tests/</code>	Текстовый отчёт прямо в консоли (по умолчанию)
<code>pytest --cov=src tests/ --cov-report=html</code>	HTML-отчёт в директории <code>htmlcov</code> — удобно смотреть в браузере
<code>pytest --cov=src tests/ --cov-report=xml</code>	XML-отчёт — для интеграции с CI/CD (Continuous Integration / Continuous Deployment), например Jenkins или GitLab